AI Concepts in DevOps · cheat sheet

DevOps Course · Session 6 · 1.3.27 · Noam Guy and Elhay Efrat

The agent in one picture

Agent = model + tools + loop + goal. **Model** decides, **context** is everything it sees on this call, **harness** runs the loop, executes tools and enforces rules.

1. Send the context to the model
2. The model replies with a tool call or a final answer
3. The harness checks permissions and runs the tool
4. The output is appended to the context; repeat

Stop conditions: goal done, max steps, budget or time limit, needs a human.

Context is a budget

- Every step re-sends the whole history: total input ≈ `N·S + (T+O)·N(N−1)/2`.
- Send slices, not dumps: `kubectl logs --tail=200 --since=1h`, `grep` before `cat`.
- Prompt caching bills the reused prefix at the cache-read price. Keep the prefix stable: no timestamps in the system prompt.
- Off track? Start a fresh session with a better first message.

Ask like you'd ask a senior colleague

What failed · where · the exact error · constraints · how to verify.

A minimal agent loop (Ollama, free and local)

```
def run_loop(messages, tools, handlers, max_steps=10) -> str:
    for _ in range(max_steps):
        response = ollama.chat(
            model="qwen3:8b", messages=messages, tools=tools, think=False,
            options={"num_ctx": 8192, "temperature": 0},
        )
        messages.append(response.message)
        if not response.message.tool_calls:
            return response.message.content
        for call in response.message.tool_calls:
            name, arguments = call.function.name, call.function.arguments
            output = handlers[name](**arguments)
            messages.append({"role": "tool", "tool_name": name,
                             "content": output})
    return "Stopped: step limit reached without an answer"
```

Append the whole message. Tools use the OpenAI-style function schema. Setup: `ollama pull qwen3:8b`, then `uv run agent.py` in the demo repo. Pin `num_ctx`: the default depends on the machine.

AGENTS.md skeleton

```
## Project
Python service, Docker, Helm, namespace: course.

## Commands
- Test: ./scripts/test.sh
- Deploy: helm upgrade --install course-app ./chart -n course

## Rules
- Run the tests before saying a task is done.
- Never run kubectl delete, helm uninstall, docker system prune.
- Images are non-root and pinned, never latest.
```

Guardrails in the harness

`agent.py` : allow, ask or deny, then a hook that can veto anything

```
ALLOW = {"get", "describe", "logs", "events", "rollout status", "rollout history"}
ASK = {"rollout undo", "rollout restart", "scale", "set"}

def permission(argv: list[str]) -> str:
    rules = {" ".join(argv[:1]), " ".join(argv[:2])}
    if rules & ALLOW:
        return "allow"
    return "ask" if rules & ASK else "deny"

def pre_tool_hook(command: str) -> str:
    event = json.dumps({"tool_name": "kubectl", "tool_input": {"command": command}})
    result = subprocess.run([HOOK], input=event, capture_output=True, text=True)
    return "" if result.returncode == 0 else result.stderr.strip() or "blocked"
```

```
#!/usr/bin/env bash
command=$(jq -r '.tool_input.command') || exit 2
blocked='--context[= ]prod|terraform.* destroy|kubectl.* delete( |$)'
if [[ "$command" =~ $blocked ]]; then
  echo "Blocked by policy: this needs a human" >&2
  exit 2
fi
```

Anything not listed is denied; "ask" needs a terminal, so in CI it means deny. Any non-zero exit from the hook blocks the call and its stderr goes back to the model. `|| exit 2` fails closed if jq is missing. A regex is a speed bump; read-only credentials are the wall. Prompts are suggestions; hooks are rules.

Read-only Kubernetes identity

```
kubectl create serviceaccount ai-agent -n course
kubectl create rolebinding ai-agent-view -n course \
  --clusterrole=view --serviceaccount=course:ai-agent
TOKEN=$(kubectl create token ai-agent -n course --duration=1h)
kubectl config set-credentials ai-agent --token="$TOKEN"
kubectl config set-context ai-agent \
  --cluster="$CLUSTER" --user=ai-agent --namespace=course
kubectl --context ai-agent auth can-i delete pods
```

The built-in `view` role reads most resources, not Secrets. The last command answers `no` .

MCP on the wire (2026-07-28)

- JSON-RPC 2.0 over stdio (one JSON object per line) or Streamable HTTP (`MCP-Protocol-Version` header).
- Stateless: every request carries `_meta` with `io.modelcontextprotocol/protocolVersion` and `clientCapabilities` . Older servers (2025-11-25 and before) use an `initialize` handshake.
- `server/discover` → `tools/list` → `tools/call` .
- Tool failure: `"isError": true` in the result, so the model can retry. Bad request or unknown tool: JSON-RPC error `-32602` .

Graders, cheapest first

1. **Deterministic**: is-json, schema, icontains, regex, helm lint
2. **Semantic**: similarity to a reference answer
3. **LLM as judge**: a specific, pass/fail rubric
4. **Human**: label a sample to check the judge

Trust a judge only after measuring agreement with human labels: Cohen's $\kappa = (p_o - p_e) / (1 - p_e)$. Above 0.8 is almost perfect.

Structured outputs (`ollama.chat(..., format=Model.model_json_schema())`) guarantee the format; evals guard the content.

Evals as a CI gate (promptfoo)

```
prompts:
  - file://../prompts/explain.txt
providers:
  - id: ollama:chat:qwen3:8b
    config: { temperature: 0, think: false }
defaultTest:
  options:
    provider:
      id: ollama:chat:qwen3:8b
      config: { temperature: 0, think: false }
  assert:
    - type: is-json
      value: { type: object, required: [summary, severity] }
    - type: llm-rubric
      value: The fix is safe and never deletes data.
tests: file://cases.yaml
```

```
- uses: $/.github/actions/setup-ollama
- name: Run evals
  env:
    PROMPTFOO_PASS_RATE_THRESHOLD: 90
  run: npx promptfoo eval -c evals/promptfooconfig.yaml -o report.html
```

Below the threshold promptfoo exits with code 100. No secrets: `setup-ollama` runs the model in the runner. Pin the model tag and `temperature: 0`; repeat flaky cases with `--repeat 3`.

Seven controls for AI jobs in CI

1. Pin actions to a commit SHA; call your own with `$/`
2. Least-privilege `permissions:`, starting from `contents: read`
3. Secrets only on the step that needs them
4. Forks get no secrets: never run PR code with secrets via `pull_request_target`
5. Event data through `env:`, never pasted into `run:`
6. Bound the cost: `timeout-minutes`, a step limit, a pass-rate gate
7. Treat AI output as untrusted: comment-only tokens, branch protection, a human merge

Six rules for AI at work

1. A human approves anything that writes
2. Read-only and least privilege by default
3. No secrets, customer data or prod credentials in prompts
4. Verify with tools: lint, test, scan, plan, dry-run
5. Review AI output like a junior's PR
6. Follow your company's AI policy

Lethal trifecta: private data + untrusted content + a way to send data out. Remove one leg.

OWASP Top 10 for LLM Applications 2025

`LLM01` Prompt Injection

`LLM02` Sensitive Information Disclosure

`LLM03` Supply Chain

`LLM04` Data and Model Poisoning

`LLM05` Improper Output Handling

`LLM06` Excessive Agency

`LLM07` System Prompt Leakage

`LLM08` Vector and Embedding Weaknesses

`LLM09` Misinformation

`LLM10` Unbounded Consumption

Try it at home

- **Free**: everything tonight runs on Ollama: `ollama pull qwen3:8b`; promptfoo talks to it as `ollama:chat:qwen3:8b`.
- **Hosted**: swap the provider line and the same evals run; pay per token.
- GitHub Models was retired in July 2026: if a tutorial uses it, swap the provider.

Read more

- `anthropic.com/engineering/building-effective-agents`
- `modelcontextprotocol.io · agents.md`
- `promptfoo.dev · github.com/confident-ai/deepeval`
- `genai.owasp.org · simonwillison.net (the lethal trifecta)`
- `docs.ollama.com · ollama.com/library/qwen3 · k8sgpt.ai`